

DOCUMENT 3

KAPISH GUPTA

3.1 Objective

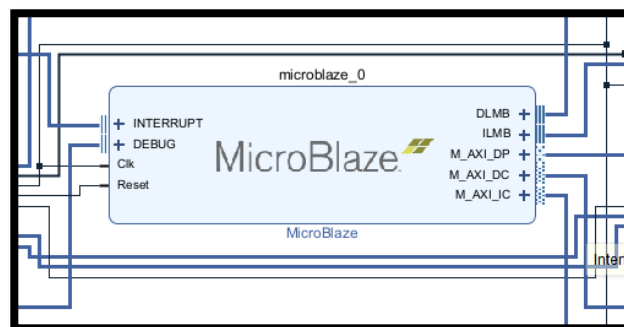
- Validate the UART serial communication via "Hello World" application in Softcore Microblaze processor.
- Verify the memory bus architecture by transferring a block of data from the processor into a Block RAM (BRAM) located in the Programmable Logic (PL), and then transferring the data into the DDR memory.

3.2 Block design creation in Vivado 2022.2

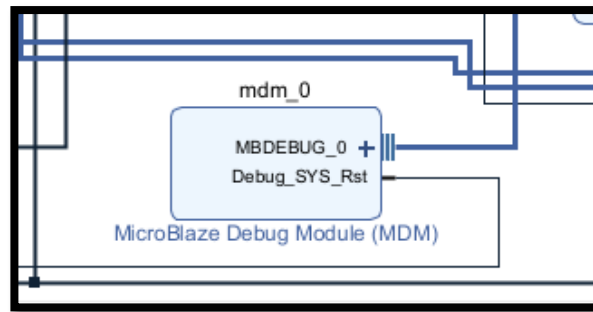
To build the MicroBlaze system, 13 specific hardware blocks (IPs) were connected in the Vivado Block Design.

Core Processor & Control

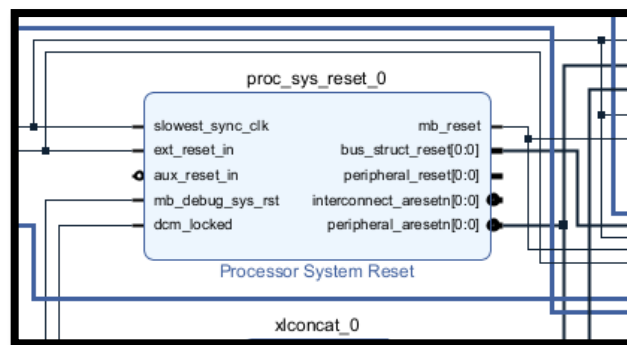
- **MicroBlaze:** A 32-bit RISC softcore processor built entirely from the FPGA's internal logic resources. It executes the C code.



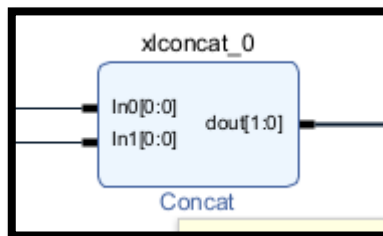
- **MicroBlaze Debug Module (MDM):** Provides JTAG debugging capabilities. It allows the Vitis software to halt the processor, read memory addresses, and step through code.



- **Processor System Reset:** Generates synchronized reset signals. It ensures all components in the design initialize in the correct sequence and at the correct clock edge.

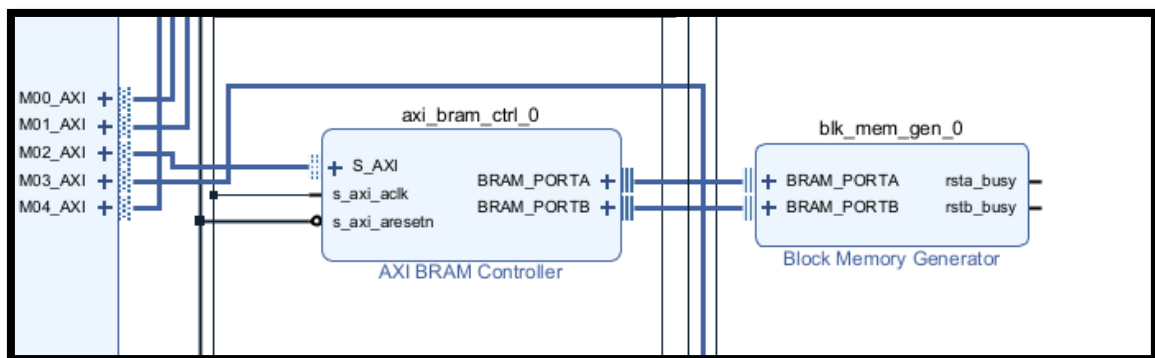
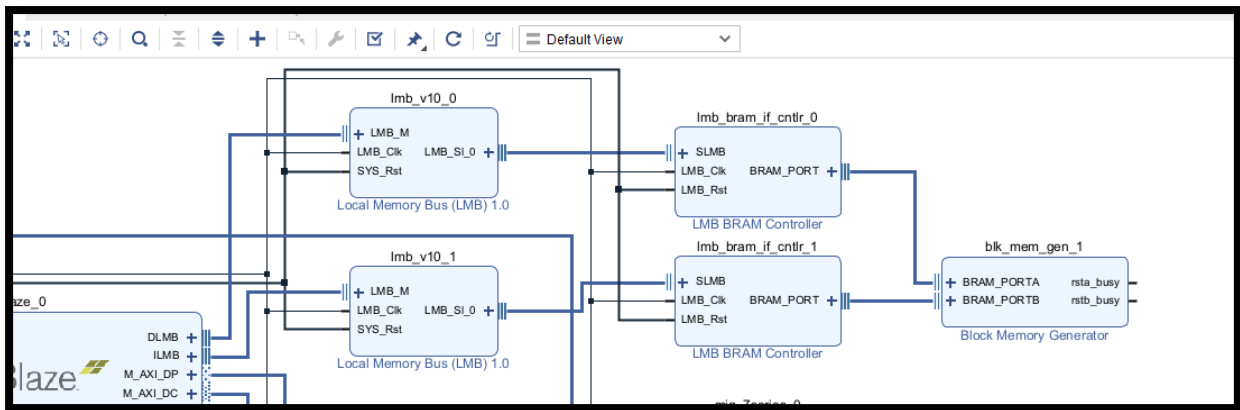


- **Constant:** Drives a fixed logic level ('1' or '0') to a specific port, typically used to permanently tie off unused pins or enable hardware features.

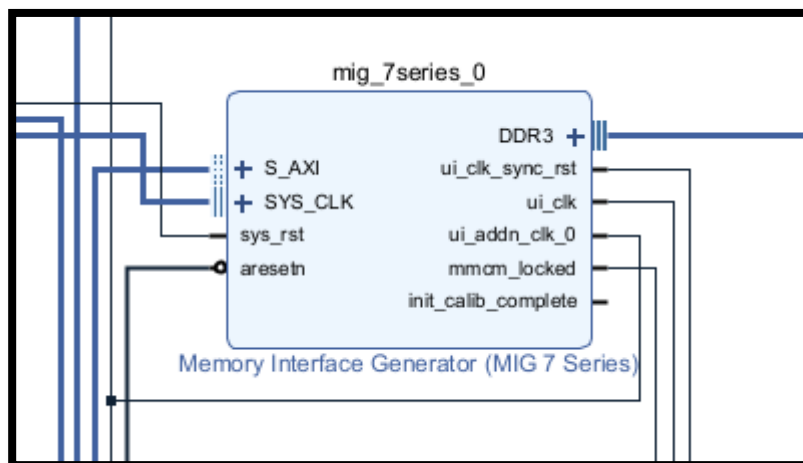


Memory Components

- **Local Memory Bus (LMB):** A fast, low-latency, point-to-point bus used exclusively by the MicroBlaze to access its closest instruction and data memory.
- **LMB BRAM Controller:** The interface module that connects the Local Memory Bus to the physical Block RAM.
- **AXI BRAM Controller:** An interface module that connects the standard AXI system bus to the physical Block RAM, allowing other system components to access the memory.
- **Block Memory Generator:** Instantiates the actual physical dual-port RAM using the FPGA's internal silicon Block RAM (BRAM) resources.

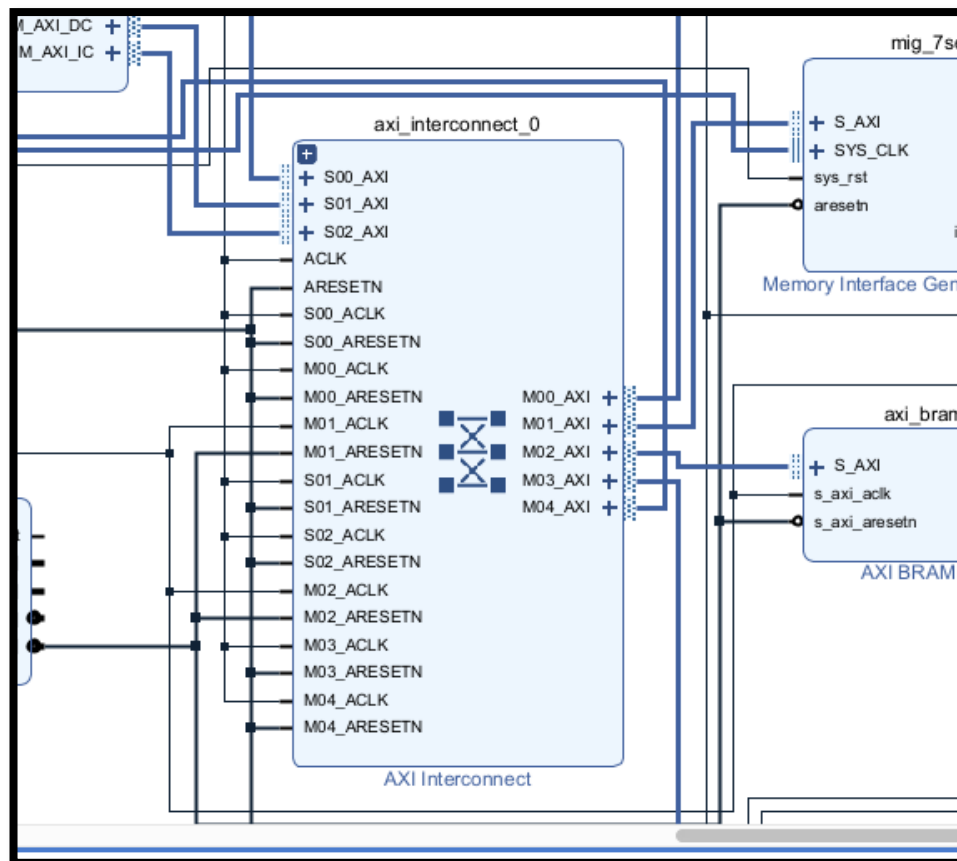


- **MIG 7 (Memory Interface Generator):** The physical memory controller. It handles the strict timing, calibration, and electrical signaling required to read and write to the external DDR3 SDRAM chips on the board.

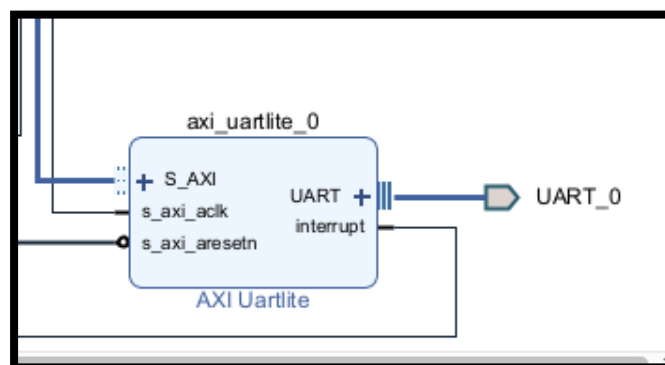


Bus Routing & Peripherals

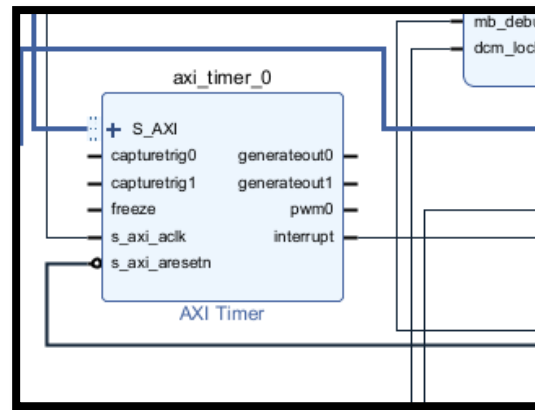
- **AXI Interconnect:** A routing block that connects AXI master devices (like the processor) to AXI slave devices (like peripherals). It manages memory addresses and data transfers across the system. It is also called crossbar switch.



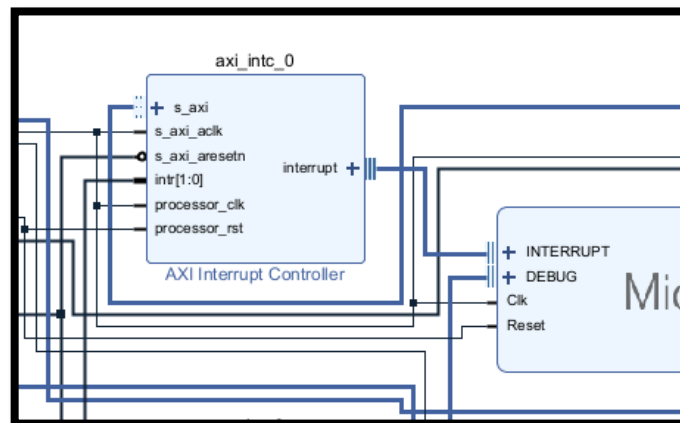
- **AXI Uartlite:** A hardware serial port. It provides basic UART communication between the FPGA and the host computer to display terminal outputs.



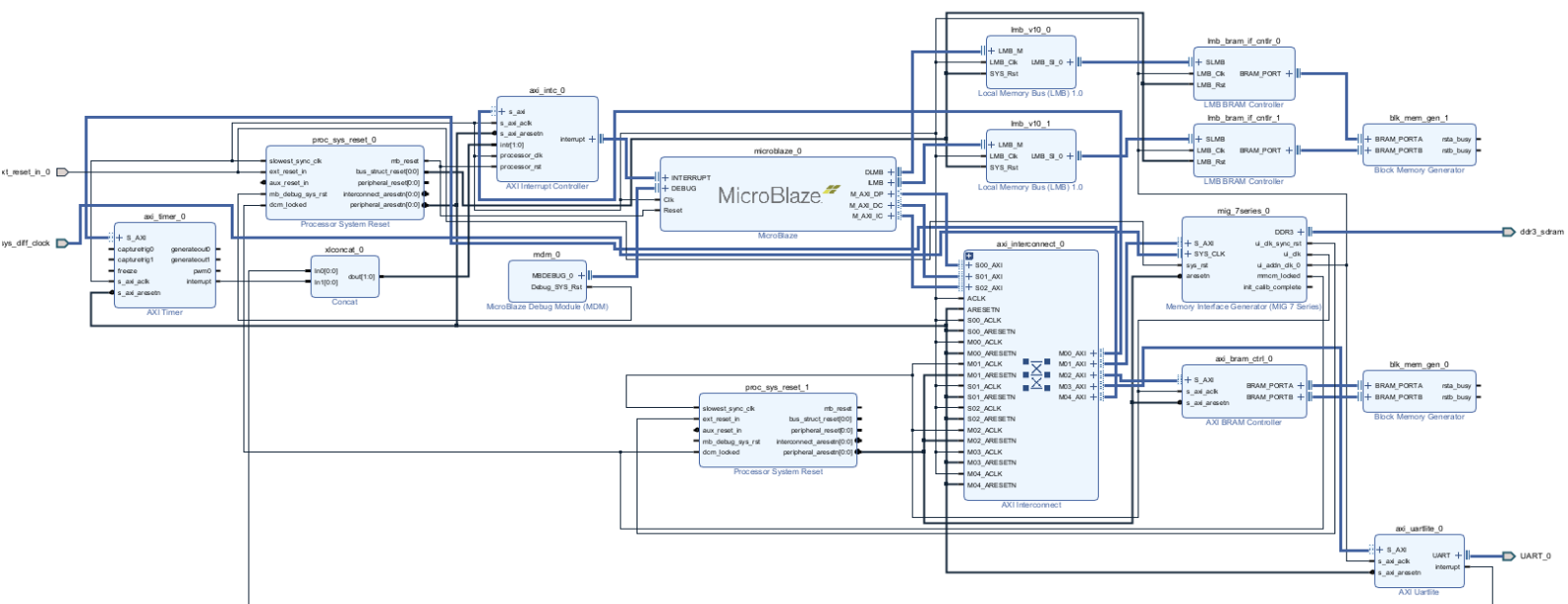
- **AXI Timer:** A 32-bit programmable hardware counter. It is used to track exact clock cycles to measure the execution time of software functions.



- **AXI Interrupt Controller:** Collects multiple hardware interrupt signals (like the signal from the AXI Timer) and routes them to a single interrupt pin on the MicroBlaze processor.



FINAL BLOCK DESIGN



- After creating block design, generate the bitstream, then export the hardware, and open Vitis.
- Make a platform project of the exported hardware file (.xsa) and then, make an application project using the platform project.
- Then save the project and Build it.
- Select the default Hello World Program.

```
*helloworld.c platform.c bram_2_ddr_m... xil_io.h xparameters.h hel
1
2 /*
3  * helloworld.c: simple test application
4  *
5  * This application configures UART 16550 to baud rate 9600.
6  * PS7 UART (Zynq) is not initialized by this application, since
7  * bootrom/bsp configures it to baud rate 115200
8  *
9  *
10 * -----
11 * | UART TYPE   BAUD RATE |
12 * -----
13 * uarts550     9600
14 * uartrlite     Configurable only in HW design
15 * ps7_uart     115200 (configured by bootrom/bsp)
16 */
17
18 #include <stdio.h>
19 #include "platform.h"
20 #include "xil_printf.h"
21
22
23 int main()
24 {
25     init_platform();
26
27     print("Hello World\n\r");
28     print("Successfully ran Hello World application");
29     cleanup_platform();
30     return 0;
31 }
32
33
34
```

```
40 * -----
41 * | UART TYPE   BAUD RATE |
42 * -----
43 * uarts550     9600
44 * uartrlite     Configurable only in HW design
45 * ps7_uart     115200 (configured by bootrom/bsp)
46 */
47
48 #include <stdio.h>
49 #include "platform.h"
50 #include "xil_printf.h"
51
52
53 int main()
54 {
55     init_platform();
56
57     print("Hello World\n\r");
58     print("Successfully ran Hello World application");
59     cleanup_platform();
60     return 0;
61 }
```

hello_world... hello_world lscript.ld helloworld.c xparameters.h xil_io.h 7

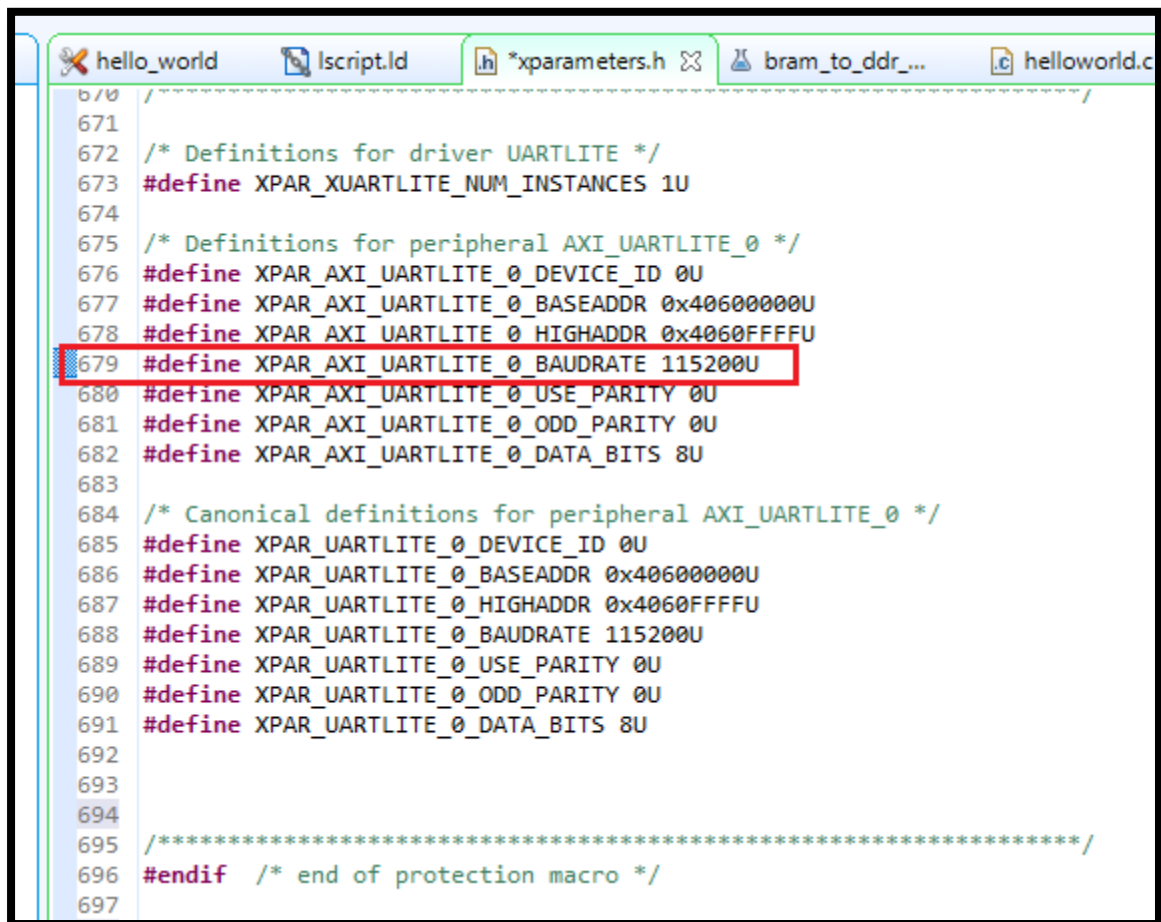
@return The 32 bit Value read from the specified input address.

Console Vitis Serial Terminal Executables Debug Shell Vitis Log Problems Debugger Console

Build Console [bram_to_ddr_Debug]

```
15:28:20 **** Incremental Build of configuration Debug for project bram_to_ddr ****
make all
make: Nothing to be done for 'all'.
15:28:20 Build Finished (took 96ms)
```

Here we can verify the baud rate of the UART in the xparameters.h file



```
670 /
671
672 /* Definitions for driver UARLITE */
673 #define XPAR_XUARLITE_NUM_INSTANCES 1U
674
675 /* Definitions for peripheral AXI_UARLITE_0 */
676 #define XPAR_AXI_UARLITE_0_DEVICE_ID 0U
677 #define XPAR_AXI_UARLITE_0_BASEADDR 0x40600000U
678 #define XPAR_AXI_UARLITE_0_HIGHADDR 0x4060FFFFU
679 #define XPAR_AXI_UARLITE_0_BAUDRATE 115200U
680 #define XPAR_AXI_UARLITE_0_USE_PARITY 0U
681 #define XPAR_AXI_UARLITE_0_ODD_PARITY 0U
682 #define XPAR_AXI_UARLITE_0_DATA_BITS 8U
683
684 /* Canonical definitions for peripheral AXI_UARLITE_0 */
685 #define XPAR_UARLITE_0_DEVICE_ID 0U
686 #define XPAR_UARLITE_0_BASEADDR 0x40600000U
687 #define XPAR_UARLITE_0_HIGHADDR 0x4060FFFFU
688 #define XPAR_UARLITE_0_BAUDRATE 115200U
689 #define XPAR_UARLITE_0_USE_PARITY 0U
690 #define XPAR_UARLITE_0_ODD_PARITY 0U
691 #define XPAR_UARLITE_0_DATA_BITS 8U
692
693
694
695 /*****
696 #endif /* end of protection macro */
697
```

Now debug and run on the hardware.

Conclusion :-

The processor was first programmed with a simple helloworld.c script. This verified that the Board Support Package (BSP) drivers were correct, the processor's clock was stable, and the UART physical pins were properly mapped to send text over the USB cable to the serial terminal.

```
47
48 #include <stdio.h>
49 #include "platform.h"
50 #include "xil_printf.h"
51
52
53 int main()
54 {
55     init_platform();
56     print("Hello World\n\r");
57     print("Successfully ran Hello World application");
58     cleanup_platform();
59
60     return 0;
61 }
62
63
```

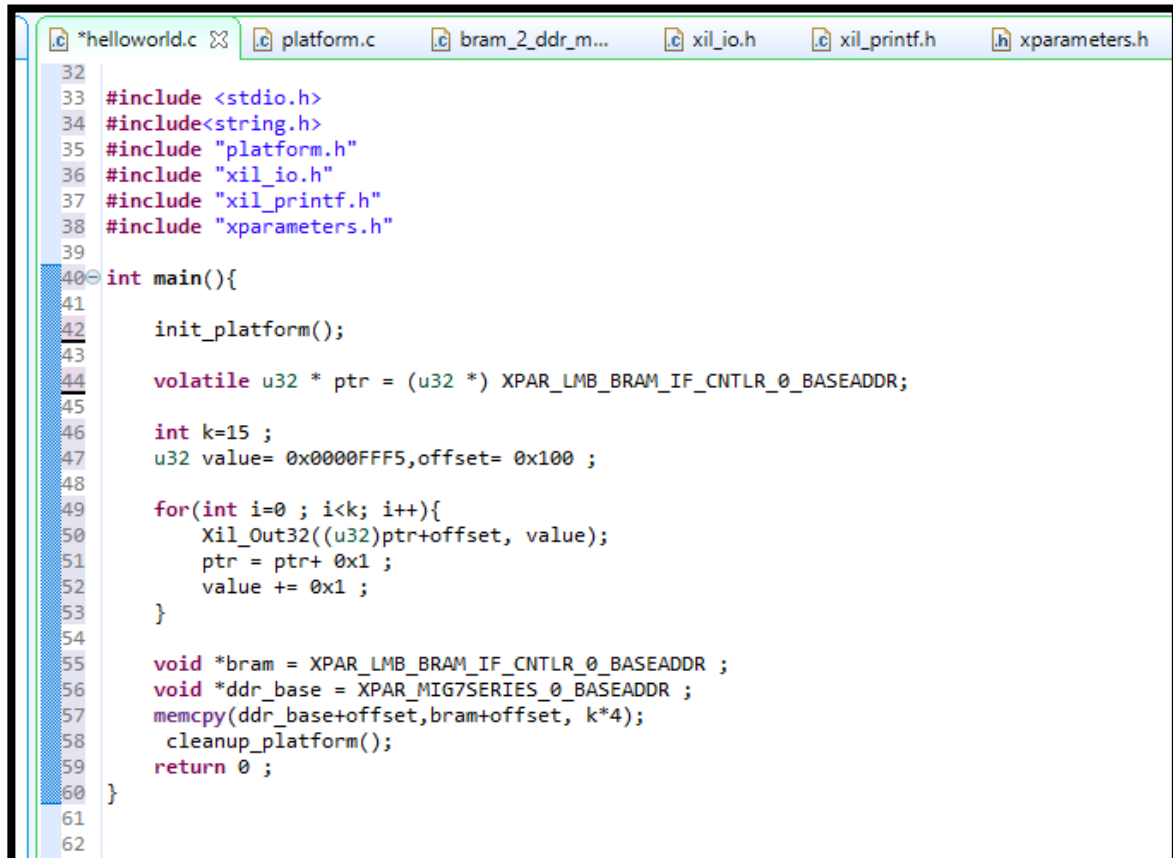
Console Vitis Serial Terminal Executables Debug Shell Vitis Log Problems Debugger Console

Connected to: Serial (COM3, 115200, 0, 8)

Hello World

Successfully ran Hello World application

Next, we tested the memory architecture. We wrote a simple for loop in C that forced the MicroBlaze processor to write data directly into the BRAM address, and then move data to the DDR memory.



```
32
33 #include <stdio.h>
34 #include <string.h>
35 #include "platform.h"
36 #include "xil_io.h"
37 #include "xil_printf.h"
38 #include "xparameters.h"
39
40 int main(){
41     init_platform();
42
43     volatile u32 * ptr = (u32 *) XPAR_LMB_BRAM_IF_CNTL0_BASEADDR;
44
45     int k=15 ;
46     u32 value= 0x0000FFF5,offset= 0x100 ;
47
48     for(int i=0 ; i<k; i++){
49         Xil_Out32((u32)ptr+offset, value);
50         ptr = ptr+ 0x1 ;
51         value += 0x1 ;
52     }
53
54     void *bram = XPAR_LMB_BRAM_IF_CNTL0_BASEADDR ;
55     void *ddr_base = XPAR_MIG7SERIES_0_BASEADDR ;
56     memcpy(ddr_base+offset,bram+offset, k*4);
57     cleanup_platform();
58     return 0 ;
59 }
60
61
62
```

Here, in the xparameters.h we can see the base address and high address of the BRAM and DDR memory.

```
*helloworld.c  xparameters.h  platform.c  bram_2_ddr_m...  xil_io.h
505 #define XPAR_LMB_BRAM_IF_CNTL0_DEVICE_ID 1U
506 #define XPAR_LMB_BRAM_IF_CNTL0_DATA_WIDTH 32U
507 #define XPAR_LMB_BRAM_IF_CNTL0_ECC 0U
508 #define XPAR_LMB_BRAM_IF_CNTL0_FAULT_INJECT 0U
509 #define XPAR_LMB_BRAM_IF_CNTL0_CE_FAILING_REGISTERS 0U
510 #define XPAR_LMB_BRAM_IF_CNTL0_UE_FAILING_REGISTERS 0U
511 #define XPAR_LMB_BRAM_IF_CNTL0_ECC_STATUS_REGISTERS 0U
512 #define XPAR_LMB_BRAM_IF_CNTL0_CE_COUNTER_WIDTH 0U
513 #define XPAR_LMB_BRAM_IF_CNTL0_ECC_ONOFF_REGISTER 0U
514 #define XPAR_LMB_BRAM_IF_CNTL0_ECC_ONOFF_RESET_VALUE 1U
515 #define XPAR_LMB_BRAM_IF_CNTL0_WRITE_ACCESS 2U
516 #define XPAR_LMB_BRAM_IF_CNTL0_BASEADDR 0x00000000U
517 #define XPAR_LMB_BRAM_IF_CNTL0_HIGHADDR 0x00001FFFU
518 #define XPAR_LMB_BRAM_IF_CNTL0_S_AXI_CTRL_BASEADDR 0xFFFFFFFFU
519 #define XPAR_LMB_BRAM_IF_CNTL0_S_AXI_CTRL_HIGHADDR 0xFFFFFFFFU
520
521
522 /* Definitions for peripheral LMB_BRAM_IF_CNTL1 */
523 #define XPAR_LMB_BRAM_IF_CNTL1_DEVICE_ID 2U
524 #define XPAR_LMB_BRAM_IF_CNTL1_DATA_WIDTH 32U
525 #define XPAR_LMB_BRAM_IF_CNTL1_ECC 0U
526 #define XPAR_LMB_BRAM_IF_CNTL1_FAULT_INJECT 0U
527 #define XPAR_LMB_BRAM_IF_CNTL1_CE_FAILING_REGISTERS 0U
528 #define XPAR_LMB_BRAM_IF_CNTL1_UE_FAILING_REGISTERS 0U
529 #define XPAR_LMB_BRAM_IF_CNTL1_ECC_STATUS_REGISTERS 0U
530 #define XPAR_LMB_BRAM_IF_CNTL1_CE_COUNTER_WIDTH 0U
531 #define XPAR_LMB_BRAM_IF_CNTL1_ECC_ONOFF_REGISTER 0U
532 #define XPAR_LMB_BRAM_IF_CNTL1_ECC_ONOFF_RESET_VALUE 1U
533 #define XPAR_LMB_BRAM_IF_CNTL1_WRITE_ACCESS 2U
```

```
*helloworld.c  xparameters.h  platform.c  bram_2_ddr_m...  xil_io.h
656
657 /* Definitions for peripheral MIG_7SERIES_0 */
658 #define XPAR_MIG_7SERIES_0_BASEADDR 0x80000000
659 #define XPAR_MIG_7SERIES_0_HIGHADDR 0xBFFFFFFF
660
661
662 /*****
663
664 /* Canonical definitions for peripheral MIG_7SERIES_0 */
665 #define XPAR_MIG7SERIES_0_DEVICE_ID XPAR_MIG_7SERIES_0_DEVICE_ID
666 #define XPAR_MIG7SERIES_0_DDR_ROW_WIDTH 14U
667 #define XPAR_MIG7SERIES_0_DDR_COL_WIDTH 0U
668 #define XPAR_MIG7SERIES_0_DDR_BANK_WIDTH 3U
669 #define XPAR_MIG7SERIES_0_DDR_DO_WIDTH 64U
670 #define XPAR_MIG7SERIES_0_BASEADDR 0x80000000U
671 #define XPAR_MIG7SERIES_0_HIGHADDR 0xBFFFFFFFU
672
673
674 /*****
675
676 /* Definitions for driver UARTLITE */
677 #define XPAR_XUARTLITE_NUM_INSTANCES 1U
```

After that, we will run our code and see the output on the corresponding address on the memory monitor to verify that data is correct.

Writing data from MicroBlaze to BRAM.

The screenshot displays a debugger interface with two main panels. The top panel, titled '(x)= Variables', shows a list of variables with their types and values. The bottom panel, titled 'Memory', shows a list of memory addresses on the left and a detailed view of the selected address (0x00000100) on the right.

Name	Type	Value
> ptr	volatile u32 *	0x0000003c
(x)= k	int	15
(x)= value	u32	0x00010004
(x)= offset	u32	0x00000100
bram	void *	0xc67d097a
ddr_base	void *	0x0fe75492

Address	0 - 3	4 - 7	8 - B	C - F
00000100	0000FFF5	0000FFF6	0000FFF7	0000FFF8
00000110	0000FFF9	0000FFFA	0000FFFB	0000FFFC
00000120	0000FFFD	0000FFFE	0000FFFF	00010000
00000130	00010001	00010002	00010003	90C60041
00000140	90C60041	90C60041	90C60041	90C60041
00000150	90C60041	90C60041	90C60041	90C60041
00000160	90C60041	90C60041	90C60041	10C62000
00000170	90C60001	BC06002C	B0000000	30600000
00000180	BC030020	3021FFE4	F9E10000	99FC1800
00000190	80000000	E9E10000	B60F0008	3021001C
000001A0	B60F0008	80000000	B0008000	E0600148
000001B0	BC2300D0	B0008000	3080000C	B0008000

Transferring the data from BRAM to DDR memory.

Variables

Name	Type	Value
ptr	volatile u32 *	0x0000003c
(x)= k	int	15
(x)= value	u32	0x00010004
(x)= offset	u32	0x00000100
bram	void *	0x00000000
ddr_base	void *	0x80000000

Memory

Monitors

0x80000100 : 0x80000100 <Hex Integer>

Address	0 - 3	4 - 7	8 - B	C - F
80000100	0000FFF5	0000FFF6	0000FFF7	0000FFF8
80000110	0000FFF9	0000FFFA	0000FFFB	0000FFFC
80000120	0000FFFD	0000FFFE	0000FFFF	00010000
80000130	00010001	00010002	00010003	00000000
80000140	00000000	00000000	00000000	00000000
80000150	00000000	00000000	00000000	00000000
80000160	00000000	00000000	00000000	00000000
80000170	00000000	00000000	00000000	00000000
80000180	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF